

INFOMCV Assignment 1

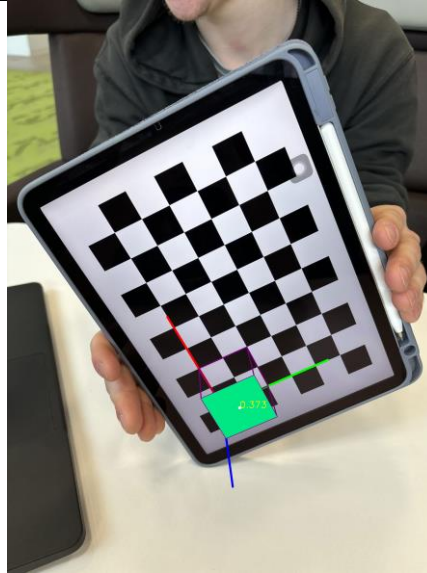
Majo Bednár (2439905), Angel Bu Tong Mei (9682120) (group 98)

Calibration

Run 1:

$$K = \begin{bmatrix} 5242 & 0 & 2147 \\ 0 & 5137 & 3074 \\ 0 & 0 & 1 \end{bmatrix}$$

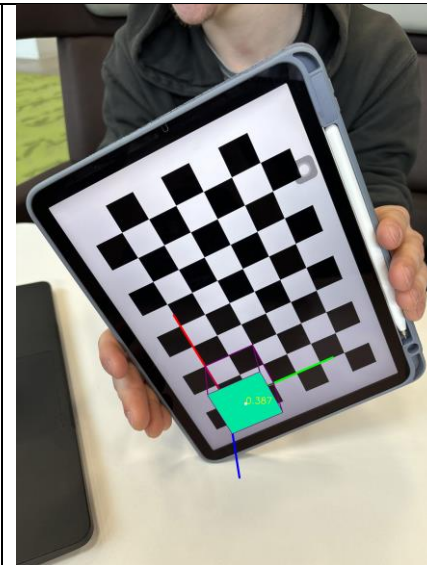
Image resolution = 4284x5712 (width x height)



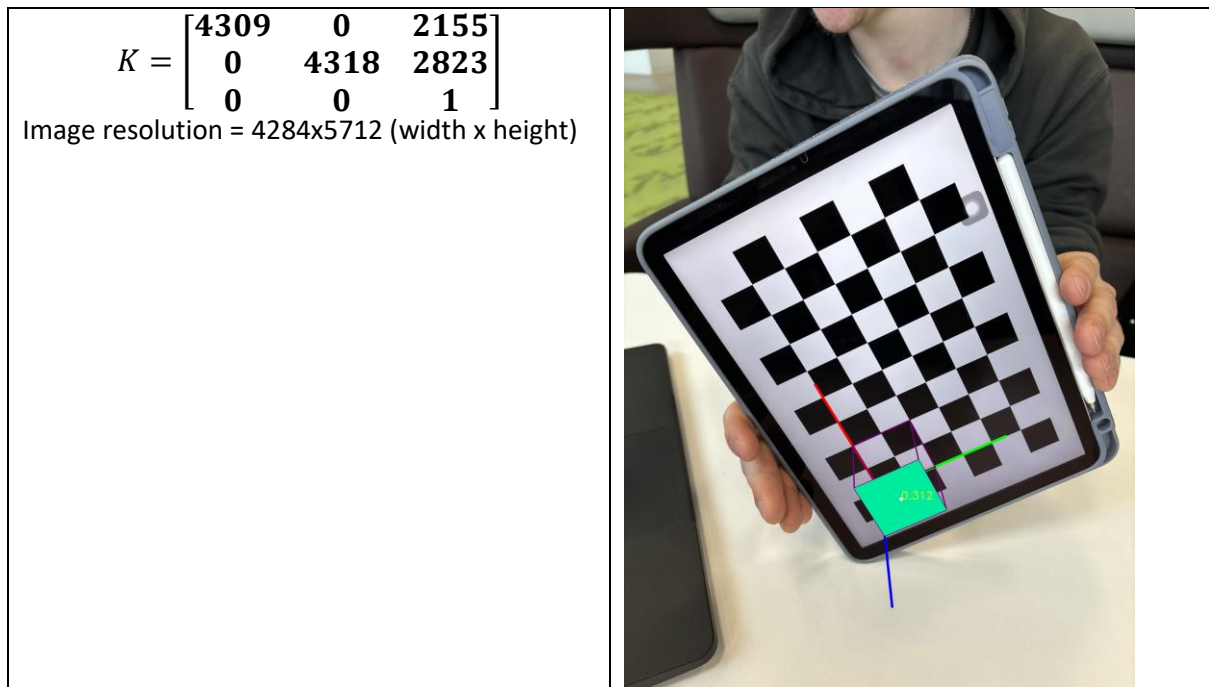
Run 2:

$$K = \begin{bmatrix} 5478 & 0 & 2223 \\ 0 & 5340 & 3019 \\ 0 & 0 & 1 \end{bmatrix}$$

Image resolution = 4284x5712 (width x height)



Run 3:



Discussion of the three runs

The focal lengths are quite similar for the 1st and 2nd run, but decrease drastically for the last run. This is probably the result of not including the pictures with manually annotated corners. Also, the horizontal and vertical focal lengths are almost the same in the last run, which indicates that they were estimated best.

The horizontal coordinate of the principal point is similar in the first and last run, and slightly higher in the 2nd run. The vertical coordinate point is similar in the first two runs, and lower in the last run. The principal point in the last run is the closest to the image centre, which is a good sign of a good calibration.

Based on what was mentioned above and re-projection errors, it seems that the best calibration resulted from the last run (RMS: 0.95 pixels), followed by the first run (RMS: 12.97 pixels), and finally the second run (RMS: 19.46 pixels). This is most likely because the images with manually annotated and linearly interpolated corners are not precise and thus worsen the calibration. The first run performed better than the second because it included more images, leading to better estimation of parameters during calibration.

Explanation of the calculation of hue and value

To calculate the intensity, we first calculate the distance of the point to camera. This is done by first calculating the extrinsic camera parameters from the intrinsic camera parameters, the chessboard corner points in the 2D image plane and in the 3D world. Then, we use these to calculate the 3D coordinates of the central point in the polygon from the camera's perspective. Since camera's coordinates in the perspective are (0, 0, 0), we simply calculate the Euclidean distance to the point. Finally, we linearly scale the intensity values from 4 to 0 meters $((4 - \text{distance}) / 4 * 255)$.

To calculate the hue, we do similar scaling based on the angle. To get the angle, we first get normal vector going from the chessboard plane ((0, 0, -1) in 3D world coordinates). Then we use the rotation vector from extrinsic camera parameters to view this normal vector from the camera's perspective. Then we can simply calculate the angle between this normal vector and the normal vector coming from the camera (0, 0, 1) by calculating the dot product of the two vectors. This gives us the cosine value of the angle, so we take arc cos of this value to get the final angle.

Choice tasks

We did choice tasks 1, 2, and 3.

For choice task 1, we applied the functions from the online phase that draw axes cube, etc. This is done for every frame the webcam captures. Given that we use the automatic detection of corners, the drawings are only displayed when it succeeds. Here is the video link:

https://drive.google.com/file/d/1WvtH4qJ_OTM9AOil4hCm4zVUvUAnrTsP/view?usp=sharing

For choice task 2, we implemented a recursive rejection of low-quality images. First, a calibration with all images is computed. Then a calibration without an image was performed for all images. For an image without which the calibration is the worst, and if it is worse than the original calibration by a threshold (epsilon), then the image is discarded and the function recurses. Otherwise, the function returns the last calibration. The threshold is necessary because a calibration without 'the lowest quality' image always has lower reprojection error. When the threshold was set to 0.2, all images with manually annotated and linearly interpolated corners were discarded. In run 1, the RMS improved from 12.97 to 0.99. In run 2, the RMS improved from 19.46 to 0.95 (so to the same reprojection error as in run 3 by default).

For choice task 3: for images where automatic detection of corners failed, we utilized perspective wrapping. We manually localized the 4 outer corners to compute a homography, warped the board to a rectified "top-down" view to detect inner corners automatically for sub-pixel precision, and warped them back to the original coordinates.

Any GenAI prompt used

We used ChatGPT for some general questions (not sure if all is mentioned, sometimes a google search results in Gemini generated answers):

- how to do vector matrix multiplication in numpy (used for computing distance)
- how to resize an image (so all chessboard corners are visible and can be clicked, and also because we later found out that some images had smaller resolution for some reason)
- What is reprojection error (to better understand how to discard bad images)
- Explain how to use `cv.getPerspectiveTransform` to flatten a chessboard image so I can detect corners more accurately than manual clicking (Issue related to choice task 3)
- How does reshape in numpy works? (Mostly regarding the interface for manual selection of corners, so that the ordering of the corners is the same as in the automatic function; also came in handy later)